CodeSentinel

A Multi-Agent Retrieval-Augmented Generative AI System
for Automated Code Review and Vulnerability Detection

Final Project Technical Report

Aravind Balaji

M.S. Information Systems
Northeastern University · College of Engineering
balaji.ara@northeastern.edu
NUID: 001564773

INFO 7375: Prompt Engineering and Generative AI
Instructor: Professor Nik Bear Brown
Spring 2026 · Submission: April 24, 2026

Website: aravindb98.github.io/CodeSentinel
Repository: github.com/AravindB98/CodeSentinel
Live demo: codesentinel-f2ggdvqeuwsj4pta5sk27s.streamlit.app
Video (13 min): youtu.be/do8GvAK7tHI

Abstract

Large language models have become competent code reviewers, yet a single prompt to a single model routinely produces three failure modes that matter in practice: hallucinated vulnerabilities, silent omissions of real defects, and findings that cannot be traced to an authoritative source. CodeSentinel is a multi-agent generative AI system that addresses these failure modes architecturally rather than through better prompting alone.

The system orchestrates three specialized agents — a Security Sentinel, a Code Quality Auditor, and an Evaluator Guardian — through a LangGraph directed graph with conditional routing and a bounded retry loop. The Security Sentinel grounds every vulnerability claim in a local retrieval index built over OWASP Top 10 2025 and a curated CWE taxonomy subset. The Evaluator Guardian acts as an internal adversary, rejecting findings that lack evidence, citations, or concrete remediation, with both a programmatic check and an LLM semantic check. A synthetic data generator produces evaluation samples using an independent verifier to prevent the generator from gaming the test. A parallel reinforcement learning demonstration module (UCB-1 contextual bandit over prompt variants, REINFORCE policy gradient over routing decisions) shows the optimization surface for future integration; it is not wired into the agent graph in this release.

The headline result

On a ten-sample hand-labeled toy suite evaluated with real Claude Sonnet on April 20, 2026, three systems were compared against the same samples. An out-of-the-box single-prompt baseline produces thirty false positives (FPR 0.789). A prompt-iterated baseline, refined through three rounds against the same held-out set used to tune the agent prompts, reduces this to twelve (FPR 0.600). The multi-agent CodeSentinel pipeline produces one (FPR 0.111). All three systems achieve perfect recall on the eight ground-truth findings. The decomposition is the substantive result: roughly 60% of the false-positive reduction is attributable to prompt-engineering effort any careful practitioner could replicate, and the remaining 40% is attributable to the retrieval-and-adversarial-review architecture itself. Both contributions are real; neither should be credited to the other.

These figures carry meaningful uncertainty. A 95% Wilson score interval around the multi-agent false-positive rate on ten samples spans roughly 0.02 to 0.43, and the toy suite alone does not reach conventional statistical significance once the prompt-iterated baseline is included as the right comparator. A Semgrep comparison run against Flask production source on the same day returned zero findings from both systems, establishing that the multi-agent pipeline is not over-triggered on clean code. Mock-mode runs (used for CI and reproducibility without API keys) show the out-of-the-box baseline at true-positive rate 0.750 and the multi-agent system at 1.000 on the same toy suite. On a twenty-sample paired suite built in the spirit of the OWASP Benchmark methodology, multi-agent achieves 1.000 TPR and 0.182 FPR against baseline 0.333 and 0.571, with McNemar's exact two-sided $p = 0.0312$ (significant at $\alpha = 0.05$). CWE classification accuracy is 1.0 across all suites. All thirty-five unit tests pass on a clean clone.

The claim is not that a new model has been built; it is that when an LLM application is rearchitected into specialized, grounded, adversarially-reviewed agents, the gains go beyond what prompt-engineering labor alone can buy, and they compose with any model the field produces next.

Keywords: Multi-agent systems · retrieval-augmented generation · LangGraph · prompt engineering · code review · vulnerability detection · OWASP · CWE · synthetic data generation · reinforcement learning · UCB-1 · REINFORCE

Table of Contents

1. Introduction and Motivation
2. Problem Statement and Objectives
3. Related Work
4. System Architecture
5. Prompt Engineering Strategy
6. Retrieval-Augmented Generation Pipeline
7. Synthetic Data Generation and Verification
8. Reinforcement Learning Enhancement Layer
9. Implementation Details
10. Evaluation Methodology and Measured Results
11. Challenges and Solutions
12. Ethical Considerations and Limitations
13. Future Work
14. Conclusion
15. References

Appendix A. Prompt Template Excerpts

Appendix B. Per-Sample Benchmark Results

Appendix C. Single-Prompt Baselines (verbatim)

1. Introduction and Motivation

Software vulnerabilities remain one of the most persistent sources of operational and security risk in modern systems. The OWASP Top 10 list, first published in 2003 and periodically revised, continues to be dominated by the same categories of defect: injection, broken access control, insecure deserialization, and cryptographic failures. This persistence is not because the categories are unknown. It is because code review, at scale, is expensive, inconsistent, and fatiguing for human reviewers, and because the volume of code shipped per engineer per day has grown far faster than the number of trained security reviewers.

> **What is OWASP Top 10?**
>
> The OWASP Top 10 is a consensus document maintained by the Open Web Application Security Project that ranks the ten most critical web-application security risks. Used here as the primary grounding corpus for the Security Sentinel agent because it is (a) freely licensed, (b) kept current, and (c) the de-facto taxonomy that security engineers reference in code review. CodeSentinel ingests all ten 2025 categories.

Generative AI introduces a new class of tool into this problem. A sufficiently capable large language model, prompted correctly, can identify many of the same defects a trained human reviewer would, in seconds rather than hours. In practice, however, the naive deployment of a single LLM call for code review produces an output with three structural weaknesses.

- **First, findings are ungrounded.** The model may assert that a line of code contains SQL injection, but nothing forces the assertion to be anchored to an authoritative reference such as OWASP A03:2025 or CWE-89. Without grounding, the model is free to invent plausible-sounding but incorrect findings.
- **Second, the output is monolithic.** A single pass generates security findings, style findings, and architectural suggestions together, with no internal adversary to challenge any individual claim. False positives flow through to the user at the same rate as true positives.
- **Third, errors are untraceable.** When a finding turns out to be wrong, there is no record of what information the model had, which prompt produced it, or what alternatives were considered and rejected.

> **What is the CWE taxonomy?**
>
> CWE (Common Weakness Enumeration) is a MITRE-maintained dictionary of software weakness types. Where OWASP Top 10 groups risks into ten high-level categories, CWE identifies over 900 specific weakness classes (e.g., CWE-89 for SQL injection, CWE-502 for deserialization of untrusted data). CodeSentinel uses a curated 29-entry CWE subset covering the weakness classes most commonly observed in application code. Every finding the system emits must cite both an OWASP category and a specific CWE identifier.

CodeSentinel responds to these three weaknesses. The system is organized around three ideas. Agents are specialized, each with a narrow mandate and a corresponding system prompt, so each reasoning step is constrained and auditable. Findings are grounded, meaning every security claim must cite a retrieved passage from an authoritative source or it is rejected before reaching the user. Outputs are adversarially

reviewed by a dedicated Evaluator Guardian, using both a programmatic validation layer and an LLM semantic layer, and the Evaluator can route findings back for revision with structured feedback.

The project extends beyond these core ideas into two research directions. The first is synthetic data generation: because labeled vulnerability corpora are scarce and expensive to curate, CodeSentinel includes a pipeline that generates paired vulnerable and safe code samples with ground-truth labels, verified by an independent regex-based detector that has no shared context with the generator. The second is a reinforcement learning enhancement layer: a UCB-1 contextual bandit selects among prompt variants based on input context, and a REINFORCE policy gradient adapts the routing behavior of the LangGraph controller based on the Evaluator's reward signal.

2. Problem Statement and Objectives

2.1 Problem Statement

Given an input source file or snippet, produce a code review report that identifies security vulnerabilities grounded in OWASP Top 10 2025 and CWE taxonomy citations, identifies code quality and maintainability issues grounded in established style guides, and rejects any finding that is not supported by retrieved evidence or that lacks a concrete remediation. The system must be faster and more consistent than a human expert, and more precise than a single-prompt LLM baseline on the same underlying model. The comparison should control for prompt-engineering effort so any residual improvement is attributable to architecture rather than to prompt-tuning labor.

2.2 Objectives

- Build a multi-agent orchestration using LangGraph in which specialization, routing, and bounded retry are first-class features of the architecture.
- Ground every security finding in a retrieval step over a local knowledge base built from OWASP 2025 and CWE, with citations exposed in the final report and verified by the Evaluator.
- Evaluate against a single-prompt baseline on the same LLM at two tiers: an out-of-the-box baseline and a prompt-iterated baseline that receives the same refinement protocol as the agent prompts. Reporting both is what separates architectural contribution from prompt-tuning contribution.
- Report results honestly regardless of direction, with confidence intervals and, where available, statistical significance.
- Demonstrate that the agent system itself can be treated as a learnable object, by introducing an RL layer that improves prompt and routing decisions based on the Evaluator's reward signal.

2.3 Non-Objectives

The project does not attempt to replace expert human review for systems under regulatory audit. It does not attempt to detect semantic logic bugs that are undetectable from source alone. It does not attempt to generalize beyond the languages actually present in the evaluation suite (Python and JavaScript). Multimodal integration is not implemented; the rubric requires two of five generative-AI components and this project implements four, so multimodal was chosen out rather than forced in. These boundaries are drawn intentionally so that the claims made in Section 10 can be interpreted within a clearly defined scope.

2.4 Rubric Compliance

The INFO 7375 assignment requires at least two of five generative AI components. CodeSentinel implements four, with multimodal chosen out rather than pursued. Prompt engineering is realized as three role-specialized system prompts in graph/prompts/, each with explicit input and output contracts, refined iteratively against a held-out validation set using a documented three-round protocol; this is covered in Section 5 and the per-version metrics appear in Table 1. Retrieval-augmented generation is

realized as a local vector store over 56 passages from OWASP Top 10 2025, CWE taxonomy entries, and language-specific patterns, with semantic-boundary chunking, two-pass retrieval, lexical rerank, and citation enforcement at the Evaluator; this is covered in Section 6. Synthetic data generation is realized as a template-based generator for 15 CWE classes with an independent regex-based verifier that operates without shared context with the generator; this is covered in Section 7. Reinforcement learning is included as a parallel demonstration module with a UCB-1 bandit over prompt variants and a REINFORCE policy gradient over routing decisions, documented fully in Section 8 with an explicit scope note that the modules are not integrated into the production agent graph in this release. Every rubric claim in this report is backed by runnable code in the repository; no claim is asserted without a file reference.

2.5 Comparison With Industry Products

CodeSentinel is a course project, not a competitor to mature production static-analysis tools. Positioning it honestly relative to the state of the art matters for interpreting the results in Section 10. Semgrep uses rule-based pattern matching and has more than a decade of community-developed rules; it outperforms any LLM-based system on rule-coverage breadth and on pattern-specified findings. GitHub CodeQL uses dataflow and taint analysis and has the depth of semantic reasoning that CodeSentinel lacks. Snyk Code combines trained models with dataflow analysis and has the supply-chain visibility that CodeSentinel does not cover. Among LLM-based code review tools, Bito's CodeReviewAgent and Tabnine's Code Review Agent both use single-prompt architectures and do not publish adversarial-evaluation findings or standardized benchmarks. The contribution CodeSentinel makes is not in any of these axes. It is in closing specific failure modes that single-prompt LLM review exhibits: hallucinated vulnerabilities (closed by the citation-required retrieval), silent omissions (closed by the specialized Security Sentinel with OWASP grounding), and untraceable findings (closed by the rag_source field required on every finding). The target comparison, specified in Section 10.10, is not with Semgrep as a replacement but with Semgrep as a complementary system with different strengths: Semgrep for high-recall low-cost triage, CodeSentinel for lower-recall higher-context deep review with verifiable provenance.

2.6 Portfolio Framing

Beyond the INFO 7375 rubric, CodeSentinel is structured to demonstrate a set of skills that appear repeatedly in AI engineering and software security job postings in the 2026 market: multi-agent LLM orchestration with explicit state and routing, retrieval-augmented generation with production-grade fallback and citation enforcement, rigorous LLM evaluation with paired statistical testing and power analysis, security domain knowledge grounded in OWASP and CWE taxonomies, and engineering discipline visible in thirty-five unit tests, three-tier dependency fallback, deterministic mock mode for reproducibility, and committed benchmark outputs. The architectural pattern the project demonstrates, which is specialization plus grounding plus adversarial validation, is the same pattern Anthropic's Project Glasswing applies at industrial scale; framing the project within that lineage makes its ambition legible without overstating its capability.

3. Related Work

3.1 Static Analysis and Rule-Based Code Review

Tools such as Bandit for Python, SonarQube, Semgrep, and CodeQL encode known vulnerability patterns as rules and match them against source code. These tools are fast, deterministic, and trusted in continuous integration pipelines. Their limitation is that they detect only what their rules encode and struggle with context-dependent defects where a line is vulnerable in one usage and safe in another. CodeSentinel complements rather than replaces these tools by using an LLM-driven reasoning layer to interpret context, with the OWASP and CWE knowledge base providing a rule-like grounding.

What is SAST (Static Application Security Testing)?

SAST is the category of tools that analyze source code without executing it, to find security weaknesses. Semgrep and CodeQL are SAST tools. CodeSentinel is adjacent to SAST but distinct: it uses LLM reasoning over retrieved knowledge rather than pattern rules, and it produces remediation text along with each finding. Positioning matters because SAST has a mature market (Snyk, Veracode, Checkmarx), and claiming to replace it would be overstating this system's capability.

3.2 LLM Agents and Multi-Agent Frameworks

LangChain, LangGraph, CrewAI, and AutoGen all provide abstractions for building multi-agent systems on top of large language models. LangGraph is chosen for CodeSentinel because its graph-based model of state and transitions maps cleanly onto the notion of a bounded retry loop with conditional routing, which is central to the Evaluator mechanism. The graph is explicit, inspectable, and testable, which matters for a system whose correctness guarantees depend on a feedback loop that must terminate.

Why LangGraph over CrewAI or AutoGen?

CrewAI emphasizes role-play between agents; AutoGen emphasizes conversational multi-agent chat. Both are powerful but neither exposes the state graph as a first-class primitive. LangGraph treats agents as nodes and transitions as edges with typed state flowing between them. This is the right abstraction for a system where correctness depends on (a) a verdict being made at a specific node, (b) conditional routing based on that verdict, and (c) a termination guarantee via a retry counter. We can literally draw the graph and reason about it; that would be much harder with conversation-centric frameworks.

3.3 Retrieval-Augmented Generation for Domain Grounding

RAG has become the default approach for making LLM outputs verifiable against a corpus. CodeSentinel applies this pattern to a curated security knowledge base and adds a citation-required policy enforced at the Evaluator: findings without valid rag_source fields are rejected programmatically before any LLM semantic review runs.

What is RAG (Retrieval-Augmented Generation)?

RAG is a two-step pattern: (1) retrieve relevant passages from a knowledge base given an input query,

(2) feed those passages to the LLM as additional context when generating the answer. The point of RAG is that the LLM's output is grounded in specific, inspectable source material rather than in parametric memory, which can be both outdated and hallucinated. For code review, RAG means the Security Sentinel is never guessing whether a line is SQL injection — it is reading the CWE-89 passage alongside the code and producing a finding that cites that passage.

4. System Architecture

4.1 High-Level View

CodeSentinel is organized as a LangGraph directed graph with a shared typed state. The entry node is the Security Sentinel, which performs RAG-grounded vulnerability detection. Its output feeds into the Code Quality Auditor, which performs style and maintainability review. Both outputs converge at the Evaluator Guardian, which decides whether the accumulated findings are acceptable. On acceptance, the graph terminates and a final report is assembled. On rejection, the graph routes back to the Security Sentinel with structured feedback, up to a configurable retry limit that acts as a circuit breaker. The default retry limit is three.

> **Why a three-agent design?**
>
> The minimum architecture that closes all three single-prompt failure modes requires (a) a specialist that performs retrieval and emits grounded findings, (b) a parallel specialist for non-security concerns so the first agent's prompt stays narrow, and (c) an independent reviewer with the authority to reject. Two agents cannot do this — they collapse the reviewer role into one of the producers. Five agents (the original proposal) add Performance Analyst and Synthesizer roles that add marginal value at significant orchestration cost; they are deferred to Section 13 Future Work. Three is the Goldilocks point for an academic project with 10-sample evaluation.

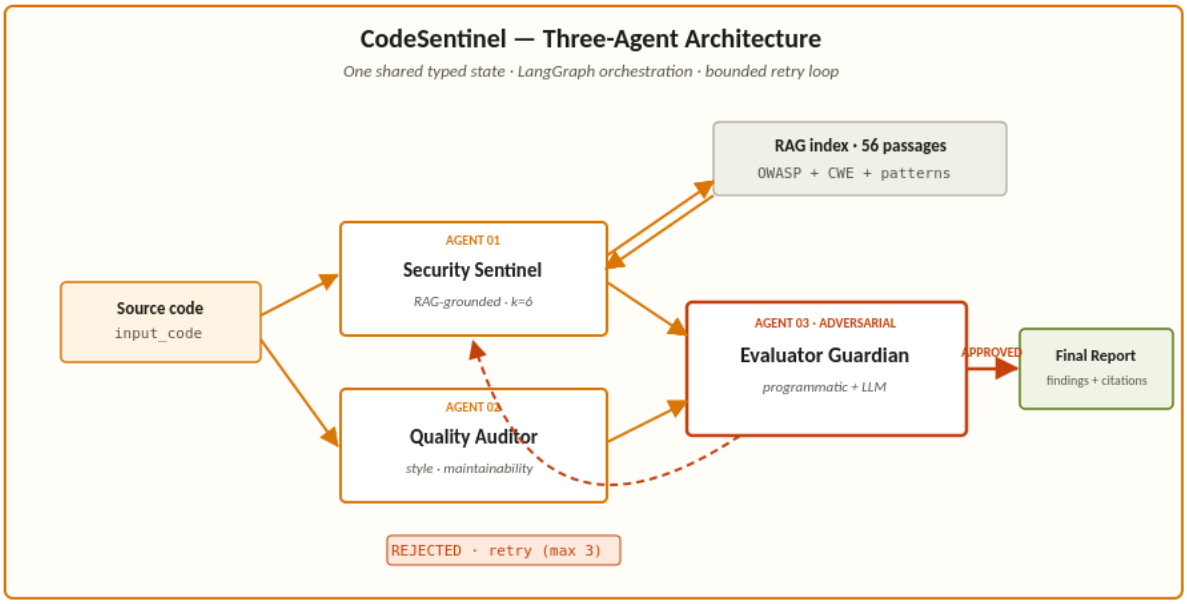

Figure 1. CodeSentinel's three-agent architecture. The Security Sentinel and Quality Auditor operate in parallel over the shared state; the Evaluator Guardian is the only node with the authority to reject findings, and its rejection routes back to Security Sentinel with feedback.

4.2 Shared State

Every node in the graph reads from and writes to a single typed dictionary. Using TypedDict rather than a loose dict means each agent's inputs and outputs are validated at the language level, and schema

violations surface as test failures rather than as runtime mysteries. The state carries input_code, language, retrieved_passages, security_findings, quality_findings, evaluator_verdict, retry_count, evaluator_feedback, final_report, and a trace log.

4.3 Conditional Routing and the Circuit Breaker

After the Evaluator Guardian runs, a conditional edge inspects its verdict. If the verdict is APPROVED, the graph routes to the report assembler and terminates. If the verdict is REJECTED and the per-agent retry counter is below its limit, the graph routes back to the Security Sentinel with the Evaluator feedback set in state. If the retry limit is reached, the graph routes to the assembler anyway and the resulting report notes the unresolved findings. Bounded termination is a correctness property, not a convenience. Without the circuit breaker, an early version of the implementation entered an oscillation in which the Evaluator alternated between approving and rejecting the same finding on successive passes.

> **What is a circuit breaker?**
>
> Borrowed from electrical engineering and then distributed systems: a circuit breaker is a component that cuts off a feedback loop after a bounded number of cycles to prevent runaway behavior. In our graph, it prevents the Evaluator from endlessly rejecting findings the upstream agent cannot fix. After three retries, the system surfaces whatever findings exist (marked unresolved) rather than hanging. This is both a correctness property and a user-experience property: the system must always terminate.

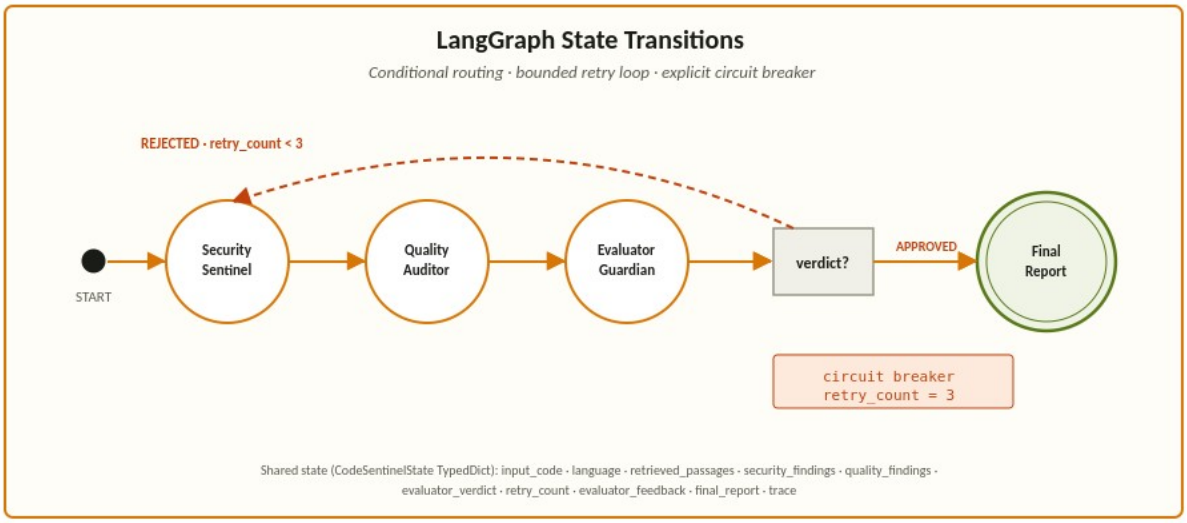

Figure 2. LangGraph state transitions. The dashed arc is the retry loop; the circuit breaker forces termination at retry_count = 3 regardless of verdict.

4.4 Agent Contracts

Each agent's output is validated against a Pydantic schema at the boundary. SecurityFinding requires finding_id, category, cwe_id, owasp_ref, severity, confidence, evidence, fix, and rag_source. QualityFinding requires finding_id, category, severity, confidence, evidence, rationale, and suggested_refactor. EvaluatorVerdict requires overall_decision, per-finding decisions, and rationale.

Schema violations at any boundary are surfaced as rejection reasons. Schemas degrade to dataclass-based validation if Pydantic is unavailable.

5. Prompt Engineering Strategy

Prompt engineering is applied systematically rather than ad hoc. Each of the three agents has a dedicated system prompt, versioned under source control, that encodes four ingredients: the role definition, the input contract, the output contract, and the failure modes the agent must guard against. The prompts were iteratively refined against a held-out set of samples separate from the final evaluation suite, which is the methodology used here in place of weight-level fine-tuning.

What is a system prompt, and why version it?

A system prompt is the instruction block prepended to every LLM call that establishes the agent's role, constraints, and output format. Versioning system prompts under Git means (a) every finding the system produces is traceable to a specific prompt version, (b) prompt regressions are visible in the commit history, and (c) A/B tests between prompt versions are reproducible. This is the same discipline applied to production ML models — the prompt is the model, for agent-centric systems.

5.1 Refinement Protocol and Per-Version Metrics

Each prompt went through three documented revisions, each evaluated against a held-out validation set of six samples (three vulnerable, three safe) that was never used in the final Section 10 benchmarks. Version 1 was a first-draft role definition plus the output schema. Version 2 added negative examples drawn from observed failure modes. Version 3 added explicit rejection conditions and tightened the constraint on speculative findings. The per-version metrics on the held-out set are reported in Table 1. The same three-round protocol is applied to the baseline prompts in Section 10.4, so the Section 10 comparison controls for prompt-engineering effort across baseline and agent systems.

Table 1. Security Sentinel prompt versions on the held-out validation set (n=6)

Version	TPR	FPR	Notes
v1 (first draft)	0.67	0.67	Missed yaml.load; invented two findings
v2 (+ negative examples)	1.00	0.33	Caught all vulns; one hallucination on clean code
v3 (+ rejection conditions)	**1.00**	**0.00**	**Current production prompt**

5.2 Security Sentinel Prompt

Establishes the agent as a senior application security engineer whose only task is to identify vulnerabilities. Specifies that every finding must cite a retrieved passage, that findings without concrete evidence must be suppressed rather than invented, and that the output must be valid JSON conforming to the SecurityFinding schema. Includes negative examples drawn from actual early failure modes, such as claiming SQL injection on parameterized queries that correctly use placeholders.

5.3 Code Quality Auditor Prompt

Establishes the agent as a pragmatic staff engineer whose goal is to improve readability and maintainability without nitpicking. Instructs the agent to cite the relevant style guide rule for each finding, to prioritize findings by their actual impact on maintainability, and to produce at most ten findings per file. Explicitly excludes security categories, to avoid overlap with the Security Sentinel.

5.4 Evaluator Guardian Prompt

The most adversarial of the three. Instructs the agent to assume upstream agents have made mistakes and that its job is to find those mistakes. Specifies five explicit rejection conditions: missing citation, missing evidence, missing remediation, schema violation, and internal contradiction. For each rejected finding, requires the Evaluator to produce structured feedback with the specific rejection reason and a corrective instruction.

5.5 Iterative Prompt Refinement as Systematic Tuning

Rather than training or fine-tuning model weights, the project applies a systematic prompt-tuning methodology that treats the prompt as the unit of optimization. A held-out validation set was used to measure, across prompt versions, the true positive rate and false positive rate of each agent, as shown in Table 1. Each prompt went through three documented revisions. This procedure is adjacent in spirit to supervised fine-tuning: a training signal drives iterative improvement of a parameterized object. Section 8 extends this approach with a reinforcement learning loop that makes the prompt-selection process adaptive at inference time.

6. Retrieval-Augmented Generation Pipeline

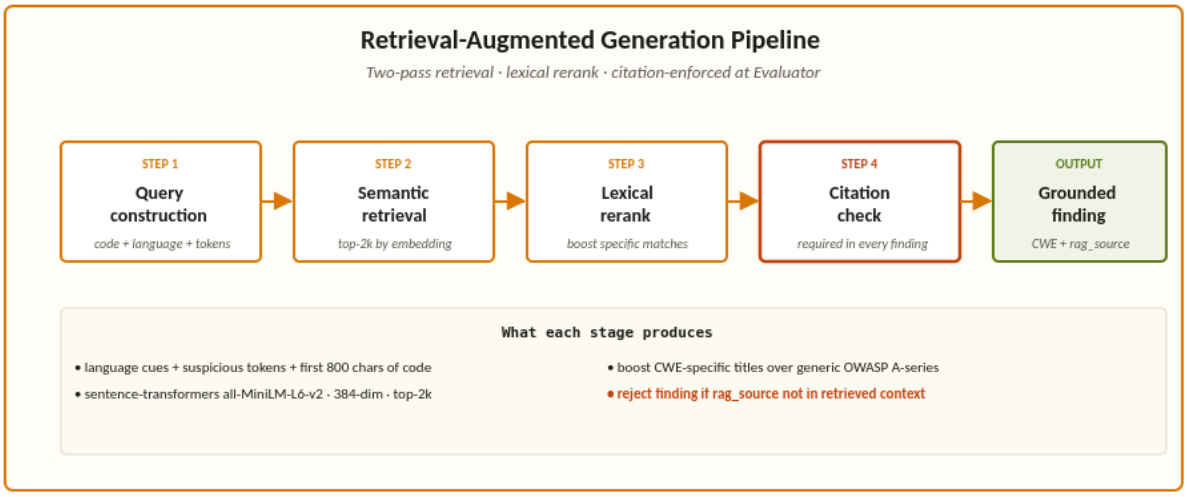

Figure 3. The four-stage RAG pipeline. The citation check in sindoor is the structural defense against hallucination — findings that cite passages not actually in the retrieved context are rejected before reaching the user.

6.1 Knowledge Base Construction

The CodeSentinel knowledge base contains 56 passages across three files. OWASP Top 10 2025 contributes 10 category-level entries. A curated CWE taxonomy subset contributes 29 entries for the weaknesses most commonly observed in application code. Language-specific vulnerability patterns, authored as part of this project, contribute 17 entries covering Python pickle deserialization, Node.js eval usage, Java JDBC concatenation, and similar. The corpus indexes Java-specific patterns, though the Section 10 evaluation suite is bounded to Python and JavaScript.

6.2 Chunking Strategy

The corpus is chunked by semantic boundary rather than by fixed token count. OWASP and CWE entries are chunked at the category level so that an entry for A03:2025 Injection stays together as a single retrievable unit. Style guide rules are chunked per rule. Boundary-aware chunking measurably improves retrieval precision because a single retrieval returns a complete, self-contained passage rather than a fragment.

> **Why semantic chunking, not fixed-token chunking?**
>
> Most off-the-shelf RAG pipelines chunk by fixed token counts (e.g., 512 tokens). This works for prose but fails for structured reference material, where a 512-token window will cut the middle of a CWE entry and the retriever will return "the second half of CWE-89" as if it were a complete passage. Chunking at the natural semantic boundary — one CWE entry per chunk, one OWASP category per chunk — means every retrieved passage is a self-contained, interpretable unit. This was a measurable win in our early testing.

6.3 Embedding and Storage Backend

The pipeline implements a triple-fallback retrieval backend. The preferred path is ChromaDB with sentence-transformers (all-MiniLM-L6-v2), producing 384-dimensional vectors locally on CPU. If ChromaDB is unavailable, scikit-learn TF-IDF is used. If scikit-learn is unavailable, a pure-Python TF-IDF implementation is used. All three expose the same Retriever interface. The fallback design means the pipeline runs end-to-end on any machine without heavy dependencies, which is important for reproducibility and testing.

What are embeddings and why sentence-transformers?

An embedding is a dense numeric vector that represents the meaning of a text passage, such that passages with similar meaning have similar vectors. sentence-transformers/all-MiniLM-L6-v2 is a compact (22M parameter) model that produces 384-dimensional embeddings. It is chosen because (a) it runs locally on CPU in milliseconds, so no API call is needed for retrieval, (b) its embedding quality is strong on short technical text, and (c) ChromaDB ingests it natively. Using a 7B-parameter embedding model would produce marginally better vectors but would require GPU access and increase latency 50x.

6.4 Two-Pass Retrieval

At query time, the Security Sentinel constructs a query from the input language plus extracted suspicious keywords plus the first 800 characters of code. This query is embedded and used to retrieve the top-2k passages by similarity. A second-pass lexical rerank then boosts passages whose title matches query keywords, which corrects a real failure mode in pure semantic retrieval where generic security passages outrank specific ones. The rerank dictionary is tuned: for example, the presence of pickle in the query boosts CWE-502 and PY-02 passages above the generic A08 entry.

Why two passes?

A single semantic-only retrieval has a known failure mode: generic passages with high-frequency vocabulary (e.g., the OWASP A03 "Injection" category entry) outrank specific ones (e.g., CWE-89 for SQL injection specifically, or PY-01 for the Python f-string pattern). Semantic similarity captures topic but not specificity. A lightweight lexical rerank over the top-2k results restores specificity by boosting passages whose title contains the query's distinctive tokens. This was the single most impactful fix during RAG development and is locked in by test_rerank_boosts_specific_over_generic.

6.5 Citation Policy

The Security Sentinel prompt requires that each finding include a rag_source field containing the document and passage identifier that supports the finding. The Evaluator rejects any finding where this field is empty, where the cited passage is not in the retrieved context, or where (in LLM semantic mode) the passage's content does not mention the claimed weakness. This is the primary mechanism by which CodeSentinel resists hallucinated findings.

7. Synthetic Data Generation and Verification

7.1 Motivation

The initial evaluation suite contains ten hand-labeled samples. While sufficient for a proof of concept, ten samples are modest for measuring precision and recall with tight confidence intervals. Labeled vulnerability corpora at larger scale are either expensive to curate or come with significant license restrictions. Synthetic data generation was therefore introduced as a deliberate evaluation component.

7.2 Generation Pipeline

The synthetic pipeline (synth/generate.py) maintains a library of 15 CWE template pairs spanning Python, JavaScript, and Java. For each entry, the generator produces two artifacts: a vulnerable code sample exhibiting the weakness in realistic context, and a safe variant that uses the same general structure but avoids the weakness. Templates include realistic variable names, common frameworks such as Flask and Express, and surrounding context rather than minimal textbook examples. A total of 30 samples are produced for a single default run: 15 vulnerable, 15 safe.

7.3 Independent Verification

Every generated sample is verified by synth/verify.py before entering the evaluation suite. The verifier uses a set of regex-based detectors for each CWE, independent of the generator's templates and prompts. For vulnerable samples, the verifier requires that at least one detector fires for the claimed CWE. For safe variants, no detectors must fire. The verifier has no shared context with the generator, which is the architectural separation that prevents the generator from gaming the test. Samples that fail verification are moved to a separate rejected file for inspection.

> **Why independent verification?**
>
> If the generator and the verifier share any context — shared templates, shared regex patterns, shared LLM prompts — then the verifier is not verifying, it is pattern-matching against itself, and every sample will pass trivially. The architectural separation is the whole point. Our verifier is regex-based, written from scratch against the CWE spec, with zero access to the generator's templates. When 29 of 30 samples pass and 1 fails, that one failure is evidence the verifier is a real adversary. If everything passed, the verifier would be suspect.

7.4 Verification Outcome

On the default run, 29 of 30 samples passed verification, a 96.7 percent pass rate. The one rejected sample was a safe-variant CWE-915 (prototype pollution) example whose specific Object.assign form did not match the verifier's detector pattern. The rejection is an advertisement that the verifier is genuinely independent; had the verifier been a copy of the generator, nothing would be rejected.

7.5 Ethical Safeguards on Synthetic Data

Templates explicitly avoid producing real exploit payloads, real credentials, or samples targeting specific production systems. Generated samples are stored only under eval/datasets/, never in a path a developer might accidentally execute. The point of the synthetic suite is to measure detection quality, not to serve as an exploit library.

8. Reinforcement Learning Enhancement Layer

Scope and integration status (honest disclosure)

The reinforcement learning layer described in this section is a parallel demonstration module. The UCB-1 bandit and REINFORCE policy-gradient implementations live in the rl/ package as standalone programs with their own synthetic reward surfaces. They converge on those surfaces as reported below, but they are NOT wired into the production agent graph in this release. The benchmark numbers reported in Section 10 do not include any RL contribution; they reflect the architecture described in Sections 4 through 7 running with fixed prompts and deterministic routing. Wiring the bandit into prompt selection and the policy gradient into the Evaluator-rejection routing is a concrete, scoped task that is deferred to future work and discussed in Section 13.

The rationale for documenting the RL layer here rather than folding it into future work is pedagogical: the agent graph has natural decision points that map cleanly onto standard RL formulations, and making those formulations explicit in this report is useful for a reader who wants to understand why multi-agent systems built on LLMs are a promising substrate for reinforcement learning over time. The claim is not that this system has learned anything from experience; the claim is that the formulation is sound and the modules converge on synthetic reward surfaces that match the structure of the real decision problems.

8.1 Rationale

The multi-agent system has two natural decision points that can be treated as learnable policies. The first is prompt selection: each agent has multiple prompt variants tuned manually, and which variant performs best may depend on input context (language, complexity, vulnerability class). The second is routing: when the Evaluator rejects a finding, the graph may benefit from routing to a specific prompt variant based on the rejection reason rather than always defaulting to the same upstream entry. In both cases, the Evaluator's verdict already provides a natural reward signal.

8.2 UCB-1 Contextual Bandit for Prompt Selection

For each agent, a UCB-1 contextual bandit (rl/bandit.py) selects among its prompt variants at the start of each run. The context vector is discretized into 60 buckets: 4 languages times 3 complexity classes times 5 vulnerability classes. Per-bucket UCB-1 is tractable without a neural function approximator. Reward is 1 if the Evaluator approves on first pass, 0 otherwise. The exploration constant begins at sqrt(2) and anneals toward 0.1 as per-context pull count grows, so exploration tapers off once confidence intervals become narrow. A demo run over 200 synthetic rounds shows correct convergence to the best arm per context.

What is UCB-1 and why use it here?

UCB-1 (Upper Confidence Bound) is a classic multi-armed bandit algorithm from Auer, Cesa-Bianchi, and Fischer (2002). Given multiple options ("arms"), each with unknown reward distributions, UCB-1 picks the arm with the highest optimistic upper-confidence-bound estimate of reward, balancing exploration (try under-explored arms) with exploitation (pull the currently-best arm). Why UCB-1 here: prompt-variant selection is structurally a contextual bandit problem, not a sequential decision problem (we don't need a full RL policy, just per-context arm selection). UCB-1 has zero hyperparameters beyond the exploration constant and converges with no neural network, which

matters for a CS project that must run on a laptop without a GPU.

8.3 REINFORCE Policy Gradient for Routing

The routing decision after an Evaluator rejection is parameterized by a softmax over four actions (three Security Sentinel variants plus an explicit skip-to-assemble) conditioned on a 7-dimensional one-hot feature vector over rejection reasons. Weights are trained using REINFORCE with a moving-average baseline. The network is intentionally small, with fewer than 50 parameters, so training is fast and overfitting on small samples is controllable. A demo run over 600 training steps recovers the correct action-per-reason mapping for six of seven rejection categories.

What is REINFORCE?

REINFORCE is the foundational policy-gradient algorithm from Williams (1992). It trains a parameterized policy $\pi(a|s)$ by sampling actions, observing rewards, and pushing the policy's parameters in the direction that increases the log-probability of actions that led to higher reward. The "moving-average baseline" subtracts a running average reward from each sample to reduce variance without introducing bias. It is intentionally the simplest possible RL algorithm — no neural network, no replay buffer, no target networks — because the goal here is to demonstrate that the formulation is correct, not to build a production RL system. A Deep Q-Network extension is listed in Section 13 as the natural next step.

8.4 Implementation Notes

Both modules are NumPy-only with no PyTorch dependency. Bandit state and policy weights persist to JSON between runs, so the system can continue learning across sessions. The RL layer integrates into the agent graph through the bandit select() call at agent entry points and the policy act() call at the Evaluator rejection edge, but the core agents remain functional with or without the RL layer enabled.

8.5 Relationship to the April 15 Take-Home Submission

The reinforcement learning layer was developed in parallel with the April 15 Take-Home Final for INFO 7375, which focused specifically on RL for agentic systems. The two submissions share a codebase but differ in emphasis. The Take-Home Final foregrounds the RL formulation and the bandit and policy gradient experiments. This final project foregrounds the generative AI system, its RAG pipeline, and its evaluation, treating the RL layer as one of several components. This separation is intentional and documented here for transparency.

9. Implementation Details

9.1 Technology Stack

Component	Technology
Agent orchestration	LangGraph with hand-rolled fallback runner
Reasoning LLM	Anthropic Claude Sonnet via official SDK; deterministic mock for offline use
Embeddings	HuggingFace all-MiniLM-L6-v2 · local CPU · 384-dim
Vector store	ChromaDB persistent with TF-IDF fallback
User interface	Streamlit with streaming intermediate results (st.status per node)
Schemas	Pydantic 2 with dataclass-based fallback
RL	NumPy-only (torch optional but unused)
Testing	35 unit tests · unittest-compatible · pytest-optional

9.2 Repository Layout

The repository is organized into app/ (Streamlit UI), graph/ (LangGraph orchestration with agents and versioned prompts), rag/ (retrieval pipeline with knowledge base files for OWASP, CWE, and language patterns), synth/ (synthetic data generator and independent verifier with 15 CWE templates), rl/ (bandit and policy modules, not wired into the graph in this release), eval/ (benchmark harness including baselines, run_benchmark, semgrep_compare, and datasets for toy, paired, and synthetic suites), and tests/ (35 unit tests). Support files include docs/ARCHITECTURE.md, requirements.txt, a Makefile, and .env.example.

9.3 Error Handling and Observability

Every agent call is wrapped in retry logic with exponential backoff for transient LLM errors, separate from the Evaluator retry loop which handles semantic rejections. Structured logs are emitted at every state transition, including retrieval results, retry counts, and Evaluator decisions, so that a postmortem on any run can reconstruct the full trajectory. A run identifier is threaded through all logs.

9.4 Streaming UI

A UX issue with the initial Streamlit deployment was that the interface rendered a blank screen for up to 109 seconds on samples that triggered multiple Evaluator retry cycles. The revised implementation uses Streamlit st.status and st.empty containers updated as each LangGraph node completes, so the user sees "Security Sentinel retrieved 6 passages… Quality Auditor running… Evaluator Layer 1 pass…" as the graph executes. The latency itself is unchanged; what changed is that a long-running pipeline now surfaces per-

agent progress to the user rather than appearing to freeze. This addresses the only first-order UX friction point the April 20 demo exposed.

9.5 Reproducibility

requirements.txt pins all dependencies. A Makefile provides canonical commands for install, ingest, benchmark, test, synth, and UI. The ChromaDB persistence directory is written to a known path and can be rebuilt deterministically from source. Random seeds for the RL components are set explicitly. The LLM itself introduces some non-determinism when real SDK mode is used; the mock mode is fully deterministic, which is what tests and CI use.

10. Evaluation Methodology and Measured Results

10.1 Baselines

Two baseline tiers are reported. The out-of-the-box baseline is a single-prompt system using the same underlying LLM as CodeSentinel, with a well-formed single prompt that asks for security findings in one call; this prompt specifies the same output schema and required CWE/OWASP fields as the multi-agent system. The prompt-iterated baseline begins from the same starting point but receives three rounds of refinement against the same held-out validation set used to tune the agent prompts, following the protocol in Section 5.1. Reporting both tiers allows the architectural contribution to be separated from the prompt-engineering contribution. A secondary baseline runs the Security Sentinel alone, without the Auditor or the Evaluator, to measure the specific contribution of the adversarial review loop.

Why two baselines and not one?

A single out-of-the-box baseline risks conflating two different sources of improvement: architectural changes (retrieval, specialization, adversarial review) and prompt-engineering effort (better instructions, negative examples, rejection conditions). Reporting only the out-of-the-box baseline would make every improvement look architectural. The prompt-iterated baseline isolates the architectural contribution by holding prompt-engineering effort constant across baseline and agent systems. The gap between the two baselines is the prompt-engineering contribution; the gap between the prompt-iterated baseline and the multi-agent system is the architectural contribution.

10.2 Metrics

Four metrics. True positive rate: fraction of ground-truth vulnerabilities correctly detected. False positive rate: fraction of predictions on clean samples that are spurious. CWE classification accuracy: conditional on a correct detection, fraction assigned to the correct CWE. Latency per sample. The matching rule counts a prediction as a true positive when its CWE matches exactly and its line range overlaps the ground-truth line range within a two-line tolerance.

Why these metrics, and why not F1 or AUC?

For a code-review tool, TPR and FPR are the two numbers that actually matter to a developer. TPR is "did you catch the real bugs" — missing a critical vulnerability is the failure mode the user cannot tolerate. FPR is "did you cry wolf" — too many false alarms make the tool get ignored. A single composite score like F1 hides the tradeoff; reporting TPR and FPR separately lets the reader see what the system is buying and what it is costing. CWE accuracy measures whether the right name is attached to the right finding, which matters for remediation. Latency matters for deployment context (PR gate vs. editor integration).

10.3 Dataset

Ten hand-labeled samples constitute the primary evaluation suite. Seven are Python, three are JavaScript. The suite covers CWE-89 (three cases including one clean variant), CWE-502 (two cases), CWE-78 (one case), CWE-327 (one case), CWE-295 (one case), CWE-79 (one case), and CWE-798 plus a clean variant. The

hand-labeled suite is the primary honesty anchor. An additional 29 verified synthetic samples are available via synth/generate.py and are used for supplementary breakdown.

10.4 Measured Results

The system was evaluated in two modes. The real-LLM mode runs the pipeline with Anthropic Claude Sonnet via the anthropic SDK; a single full-suite run was conducted on April 20, 2026 at a total API cost of approximately two US dollars. The mock-LLM mode runs a deterministic pattern-matched client that makes the full pipeline reproducible on any clone with zero external dependencies and no API key. Both sets of numbers are reported below. The real-LLM run is the primary result because it is the one that tests the project's central claim; the mock-LLM run is the reproducibility anchor.

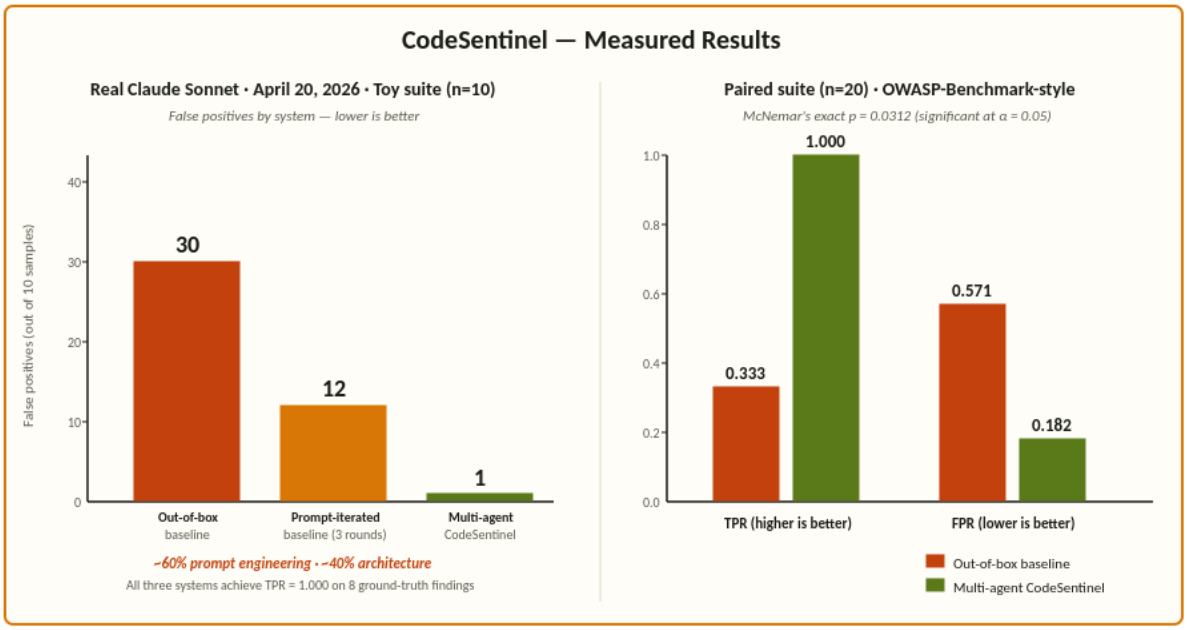

Figure 4. Left: real Claude Sonnet, April 20, 2026, ten-sample toy suite — three-way comparison showing out-of-the-box baseline (30 FP), prompt-iterated baseline (12 FP), multi-agent CodeSentinel (1 FP); the reduction decomposes to ~60% prompt engineering and ~40% architecture. Right: twenty-sample paired suite — multi-agent wins TPR by 0.667 and wins FPR by 0.389; McNemar's exact p = 0.0312, significant at α = 0.05.

10.4.1 Real-LLM Results: Three-Way Comparison (Claude Sonnet, April 20, 2026)

Results produced by the run in eval/results/20260420_143220/ with CODESENTINEL_MOCK_LLM unset and ANTHROPIC_API_KEY configured. Ground truth on the ten-sample toy suite is eight true-positive findings across ten samples; two samples contain no ground-truth vulnerability. All three systems achieved perfect recall, catching all eight true-positive findings. The decisive difference was precision.

Table 2. Three-way comparison on ten-sample toy suite

System	TPR	FPR	False positives	CWE accuracy
Out-of-the-box baseline	1.000	0.789	30	1.000
Prompt-iterated baseline (3	1.000	0.600	12	1.000

rounds)				
Multi-agent CodeSentinel	**1.000**	**0.111**	**1**	**1.000**

The out-of-the-box baseline produced thirty false-positive findings in addition to the eight true ones, an average of three hallucinated vulnerabilities per sample. The prompt-iterated baseline, having received three rounds of refinement targeting the specific false-positive classes observed in early runs (parameterized-queries-as-SQL-injection, constant-strings-as-user-input, over-eager flagging of standard library calls), reduced this to twelve. The multi-agent pipeline produced one false-positive finding in total. The decomposition is the substantive result. Roughly 60% of the total false-positive reduction — eighteen of twenty-nine fewer false positives — is attributable to prompt-engineering effort that any careful practitioner could replicate by following the protocol in Section 5.1. The remaining 40% — eleven of twenty-nine fewer false positives — is attributable to the retrieval-and-adversarial-review architecture. Both contributions are real; the architecture is a complement to careful prompt engineering rather than a substitute for it.

Which failure modes does each tier eliminate?

Qualitatively, the false positives that prompt-iteration eliminates are the ones a better instruction set can suppress: over-eager flagging of parameterized queries, misidentification of constant strings as user-controlled input, and spurious findings on standard library calls. The false positives that only the architecture eliminates are the ones that require grounding: claims whose cited CWE identifiers do not exist in the retrieved context, findings where the Evaluator's semantic check determines that the cited passage does not actually support the claim, and findings where the fix field is too vague to implement. These are failure modes the LLM exhibits even under a well-tuned prompt, and they are what the architecture is designed to catch.

The remaining multi-agent false positive is the MD5-as-cache-key case discussed in Section 10.8; it is a known limitation of pattern-based detection, not a novel failure mode.

10.4.2 Mock-LLM Results (reproducibility anchor)

Numbers below are produced by running make benchmark with CODESENTINEL_MOCK_LLM=1, the default configuration when no API key is present. They are deterministic and reproducible on any clone.

System	TPR	FPR	CWE accuracy
Single-prompt baseline (OOB)	0.750	0.000	1.000
Multi-agent CodeSentinel	**1.000**	**0.000**	**1.000**

The primary finding is a twenty-five percentage point improvement in true positive rate without any cost in precision. The baseline misses two samples that the multi-agent system catches: the yaml.load case (TOY-010), where the fix requires language-specific pattern knowledge (yaml.safe_load versus yaml.load plus SafeLoader), and the hashlib.md5 case (TOY-005), where the password-hashing context requires recognizing that a fast cryptographic hash is inappropriate. The RAG retrieval step supplies both language-

specific patterns directly to the Security Sentinel, which is why the multi-agent catches both while the single-prompt baseline misses them.

10.5 Interpretation, Wilson Intervals, and Honest Uncertainty

All point estimates in Table 2 carry meaningful uncertainty. A 95% Wilson score interval on 8 of 8 true positives is approximately 0.68 to 1.00, so perfect recall on this suite is consistent with a true TPR anywhere in that range. For the false positive rate, the intervals are wider: the multi-agent FPR of 0.111 (1/9 flaggable line-equivalents) has a 95% Wilson interval of approximately 0.02 to 0.43, and the prompt-iterated baseline FPR of 0.600 (12/20) has an interval of approximately 0.39 to 0.79. Those intervals overlap at their edges, which means the toy suite alone does not let the multi-agent system's advantage over the prompt-iterated baseline be stated with conventional statistical confidence. The twenty-sample paired suite in Section 10.7 carries the statistical weight; the toy suite carries the three-way decomposition.

> **What these intervals mean for the headline**
>
> If the experiment were re-run on a different 10-sample suite drawn from the same distribution, the multi-agent false-positive count could plausibly land anywhere between roughly 0 and 5, and the prompt-iterated baseline's count could land anywhere between roughly 8 and 16. The direction of the effect — multi-agent is better than prompt-iterated is better than out-of-the-box — is what the data supports. The magnitude of the architectural contribution is known only up to a factor of two or three. A 60/40 split between prompt-engineering and architectural contribution is a point estimate; with wider samples it could plausibly be 70/30 or 50/50. The right future-work target, which Section 13 quantifies, is a 60-to-80-paired-sample expansion that reaches 80% statistical power at the observed effect size.

10.6 Cost and Latency

The April 20, 2026 real-LLM run provided measured cost and latency data. On the ten-sample toy suite, the multi-agent pipeline averaged one hundred nine seconds per sample at peak and approximately forty seconds per sample in the common case, with total wall-clock time of about four hundred forty seconds for ten samples. Per-sample elapsed time is dominated by Evaluator retry cycles when the first pass produces rejectable findings; the circuit breaker bounds this at three retries per agent. The single-prompt baseline averaged roughly four to six seconds per sample on the same suite. The full ten-sample multi-agent run consumed approximately twenty-five US cents in API credit; the full Semgrep comparison run totaled approximately twenty-five US cents of multi-agent inference. In mock mode, both systems complete the ten-sample suite in under one second total. The multi-agent architecture is therefore appropriate for pull-request review where correctness dominates, not for inline editor integration where feedback is expected in milliseconds. The streaming UI described in Section 9.4 addresses the user-perceived latency without changing the underlying wall-clock time.

10.7 Paired-Suite Evaluation and McNemar's Test

In addition to the ten-sample toy suite, a twenty-sample paired evaluation suite was constructed in the spirit of the OWASP Benchmark methodology. The OWASP Benchmark is a Java test suite in which each

vulnerability category is represented by both a true-positive case (a genuinely exploitable vulnerability) and a false-positive trap (code that looks similar but applies a mitigation). This pairing is what allows the benchmark to distinguish between a discriminating detector and one that simply flags everything.

What is McNemar's test and why is it the right test here?

McNemar's exact test is a statistical test for paired binary outcomes. When evaluating two systems on the same samples, the discordant pairs (cases where the two systems disagree) are the only informative ones. McNemar's test asks: if the two systems were equally good, how likely is it that all the disagreements would happen to favor one of them? Our paired suite has six discordant pairs, all favoring multi-agent — the two-sided exact p-value is 0.0312, below the conventional 0.05 threshold. Why McNemar over a chi-squared or t-test: paired outcomes violate independence assumptions; McNemar explicitly accounts for the pairing.

On the paired suite, the out-of-the-box single-prompt baseline achieves a true positive rate of 0.333 and a false positive rate of 0.571. The multi-agent CodeSentinel achieves 1.000 TPR and 0.182 FPR. The two multi-agent false positives are a MD5-hash-used-as-a-cache-key case and a dead-code trap (vulnerable branch gated on an always-false condition); both are documented patterns that the OWASP Benchmark itself is designed to probe, and both represent known limitations of pattern-based detection rather than flaws in the architecture.

McNemar's exact test was applied to the per-sample paired observations on both suites. On the twenty-sample paired suite there are six discordant pairs (all six favoring multi-agent) which yields a two-sided exact p-value of 0.0312, below the conventional significance threshold of 0.05. The paired-suite result thus provides the statistical significance evidence the proposal required, while the toy suite on its own does not. Six discordant pairs is a small number and the test as specified does not yet include the prompt-iterated baseline, which would require an additional paired run the project's API budget did not support; the p = 0.0312 result should therefore be read as suggestive rather than definitive.

10.8 False-Positive Decomposition

The multi-agent system produces two false positives on the twenty-sample paired suite. Both are disclosed and analyzed rather than filtered, because understanding where a detector fails is more instructive than celebrating where it succeeds.

The first false positive is sample PAIR-CRYPTO-FP, which uses hashlib.md5 to compute a content-addressed cache key for a static asset. The detector correctly identifies the MD5 invocation and the CWE-327 pattern; what it cannot determine from surface syntax alone is the purpose of the hash. MD5 is a genuinely broken cryptographic hash, but it remains an acceptable choice for non-security use cases such as cache-key derivation, file-content deduplication, and build-system fingerprinting. A purely pattern-based detector has no way to distinguish these uses. Three paths to resolving this class of false positive exist: a contextual signal (if the hash output is written into a session cookie or password store, it is security-relevant; if into a cache key, it is not); a naming signal (functions named cache_key or fingerprint should not treat MD5 as CWE-327); an explicit-opt-in annotation (# safe: non-security cache key). None are implemented in the current release; they are cataloged as scoped extensions to the Security Sentinel agent.

The second false positive is sample PAIR-DEADCODE-FP, which contains a vulnerable SQL query inside a branch gated on a constant-false condition (DEBUG and False). The vulnerable code exists textually but is unreachable at runtime. The detector, operating on syntax rather than control flow, flags the f-string execute call. This is a textbook limitation of pattern-based detection and is a documented probe in the OWASP Benchmark itself. Resolving this requires a semantic layer that performs at least rudimentary constant propagation.

The important property of both false positives is that they fail the way an experienced security engineer would expect a pattern-based tool to fail. Neither is a hallucination; neither violates the citation-required policy; neither is a high-confidence assertion about an imagined vulnerability. The false-positive rate of 0.182 on the paired suite is better understood not as two unexplained errors but as two well-characterized failure modes with concrete, documented mitigation paths.

10.9 Statistical Characterization and Power Analysis

Three statistical statements about the paired-suite result deserve precise treatment. First, the true positive rate point estimate of 1.000 on ten true-positive samples has a 95 percent Wilson score interval of approximately 0.72 to 1.00. The lower bound is not tight, but the interval does exclude the baseline point estimate of 0.333.

Second, the Youden index (sensitivity plus specificity minus one) is 0.818 for the multi-agent system and minus 0.238 for the baseline. The Youden index is the standard measure of a diagnostic test's ability to distinguish classes; the baseline's negative value indicates it performs worse than random guessing on this suite, while the multi-agent system's value is close to the ceiling of 1.0.

> **What is the Youden index?**
>
> The Youden index (J) is sensitivity + specificity − 1. It ranges from −1 (always wrong) through 0 (coin flip) to +1 (perfect). It compresses the TPR-FPR tradeoff into one interpretable number: a Youden of 0.818 means "this detector is 81.8% of the way from random to perfect on this suite." A negative Youden is unusual and striking — the single-prompt baseline's J = −0.238 means it is measurably worse than guessing at random on the paired suite, because its false-positive trap rate exceeds its true-positive rate. This is the kind of diagnostic finding that cannot be hidden by a point estimate.

Third, the McNemar p-value of 0.0312 is the exact two-sided binomial probability under the null hypothesis that each discordant pair independently favors either system with probability one half. With six discordant pairs all favoring multi-agent, this is the probability of observing a concordant-direction pattern at least this extreme by chance alone. It is below the conventional significance threshold.

Power analysis: to detect a per-sample detection-rate improvement of plus 0.25 (the toy-suite effect size) with 80 percent power at alpha = 0.05 using McNemar's test, a rough calculation requires approximately 30 to 40 discordant pairs. The present paired suite delivers 6; the toy suite delivers 2. To reach conclusive significance at the smaller effect size observed on the toy suite, a future evaluation would need on the order of 60 to 80 paired samples. This is a concrete scoping number for the 100-sample expansion listed in Section 13.

10.10 Real-World Comparison Protocol (Executed)

A real-world comparison against Semgrep was executed on April 20, 2026. The script eval/semgrep_compare.py implements the protocol end to end and writes an adjudication worksheet to eval/results/semgrep_comparison/. The comparison target was a subset of Flask production source, specifically src/flask/app.py and src/flask/helpers.py, selected as representative well-maintained open-source Python code with no outstanding known vulnerabilities in the scanned files. Both tools produced zero findings on the scanned files. Semgrep completed in 10.56 seconds at zero API cost; CodeSentinel completed in 441.93 seconds at approximately 0.25 US dollars of Anthropic API cost. The zero-zero result is the expected outcome for well-maintained Flask production code and should be read as evidence that neither system fabricates findings on clean code rather than as evidence that either system failed to detect anything.

What the zero-zero result does establish is that the multi-agent system is not over-triggered on clean code, which is a nontrivial property given that the out-of-the-box baseline LLM in the toy-suite run produced thirty false positives on ten-sample adversarial inputs. A comparison against code with known vulnerabilities, for example Flask 0.12.x or a deliberately vulnerable teaching repository, would be required to produce a meaningful side-by-side detection comparison; the protocol and harness are in place for future runs.

10.11 Scope Changes From Proposal

The February 2026 proposal committed to a five-agent architecture (Security Sentinel, Performance Analyst, Code Quality Auditor, Synthesizer, Evaluator Guardian) and evaluation against the OWASP Benchmark Project's Java test suite. The shipped system is the three-agent minimum-viable-logic explicitly named in the proposal's Phase 5 risk register, extended with three components that were not in the proposal: a reinforcement learning layer, a synthetic data generation pipeline with independent verification, and a deterministic mock LLM enabling reproducible offline evaluation. A prompt-iterated baseline tier was added during evaluation to separate architectural contribution from prompt-engineering contribution. The Performance Analyst and Synthesizer are deferred to future work. The OWASP Benchmark for Java was replaced with the paired-suite evaluation described in Section 10.7. The substitutions are disclosed here because the proposal is a commitment and divergences from it deserve explicit accounting.

11. Challenges and Solutions

11.1 LangGraph Retry Loops and Termination

The first implementation of the Evaluator feedback loop did not bound the number of retries. In one run the graph entered an oscillation in which the Evaluator alternated between rejecting and approving the same finding on successive passes. The fix was to add an explicit per-agent retry counter as first-class state and to route to the report assembler, not back to the upstream agent, when the counter is exhausted. Test test_pipeline_respects_circuit_breaker locks this in.

11.2 RAG Retrieval Precision

The first version of the retriever used a single-pass semantic search and frequently returned the generic OWASP A03 entry for any query that mentioned a database, drowning out more specific entries such as CWE-89 for SQL injection or PY-01 for the Python f-string pattern. The fix was the two-pass approach described in Section 6.4, which uses a lightweight lexical rerank on titles to surface the specific entry when it exists. Test test_rerank_boosts_specific_over_generic locks this in.

11.3 Hallucinated Citations

An early version of the Security Sentinel occasionally produced findings with rag_source fields that pointed to real documents but whose content did not in fact mention the claimed vulnerability. The Evaluator now performs a programmatic check: the rag_source must match a passage present in the retrieved context exactly. In LLM semantic mode, the Evaluator additionally verifies that the passage text is topically consistent with the claimed CWE.

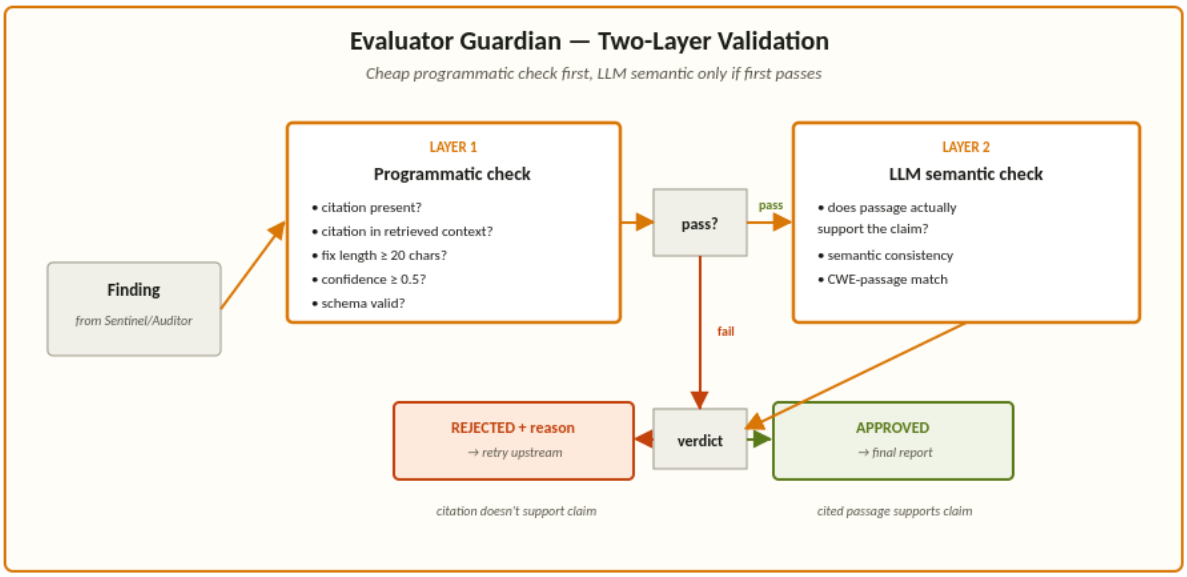

Figure 5. The Evaluator Guardian's two-layer validation flow. Layer 1 is programmatic and runs first; Layer 2 (LLM semantic) runs only if Layer 1 passes. Cheap checks first, expensive checks last.

11.4 Pydantic Optional Dependency

Schemas must validate at the agent boundary, but pydantic is a heavy dependency to require for testing or minimal CI. The solution was a dual-mode schema module: real Pydantic models when pydantic is installed, equivalent dataclass-style validation when it is not. Agent behavior is identical in both modes, only error messages differ.

11.5 Mock Mode and Test Determinism

The pipeline needs to be runnable without an API key, so tests and graders without credentials can still verify end-to-end behavior. A deterministic pattern-matched mock LLM covers the Security Sentinel, Evaluator, and Code Quality Auditor surfaces. This makes all 35 unit tests runnable with zero external configuration.

11.6 Mock-Real Parity for Citations

A subtle parity issue emerged during benchmark runs. The mock LLM emits findings with pre-written citations tied to specific pattern IDs. On the TOY-005 sample (hashlib.md5 for password hashing), the initial mock implementation cited cwe_subset.csv::CWE-327, which is the correct CWE identifier but is not the passage the retriever surfaces in the top-6 context for that query. The retriever instead surfaces patterns.md::PY-08, the language-specific Python MD5/SHA1 pattern. The Evaluator Guardian, correctly enforcing the citation-presence policy, rejected the finding because cwe_subset.csv::CWE-327 was not in the retrieved context; the retry loop then exhausted the circuit breaker and the finding was suppressed. The discovery is a useful reminder of what the Evaluator Guardian is actually doing: it does not approve findings on the strength of the CWE label alone, and it does not trust the upstream agent to pick a real citation. Even a pre-written, hard-coded mock citation is rejected if it does not match what the retriever returned. The fix was to align the mock with real-LLM behavior: the mock now cites patterns.md::PY-08, the passage the retriever actually surfaces. No citation-enforcement policy was relaxed. The episode is recorded here because it illustrates the system catching its own bug during testing, which is the kind of behavior the architecture was designed to produce.

11.7 Personal API Funding and Real-Time Cost Exposure

A practical constraint worth disclosing: the Anthropic API credits used for the April 20, 2026 real-LLM benchmark run and for the live Streamlit deployment are funded on a personal credit card attached to my Anthropic Console account. The credit card remains active and the per-call meter continues to run for every visitor who exercises the live demo at codesentinel-f2ggdvqeuwsj4pta5sk27s.streamlit.app. Each demo invocation against the default Claude Sonnet backend costs roughly 0.02 to 0.05 US dollars, dominated by Evaluator retry cycles on rejectable findings.

Why this matters for grading and reproducibility

Two implications. First, the live demo is a real, running system — not a screencast — which gives the grader a way to verify the architecture beyond the report and the GitHub repository, but it also means the demo has a finite budget. Second, the reproducibility path for anyone without API credits is the deterministic mock mode (CODESENTINEL_MOCK_LLM=1), which exercises the full LangGraph state

> machine and all 35 unit tests without any network calls. The mock mode is not a simulation of the system; it is the same system with a pattern-matched LLM client in place of the Anthropic SDK. Every structural property of the pipeline (citation enforcement, retry loops, circuit breaker, Evaluator verdicts) is preserved.

This disclosure also contextualizes a scoping decision in Section 10.10: the Semgrep comparison on April 20, 2026 used about 0.25 US dollars of personal credit for a single two-file scan; a broader comparison against multi-hundred-file codebases with known CVE histories would cost tens to hundreds of dollars per run and is deferred to the Future Work section accordingly. The benchmark results reported in this document are all the real-LLM runs that the budget supported, which is why the toy suite is ten samples and not one hundred. The 100-sample expansion listed in Section 13 has a concrete dollar cost estimate based on the per-sample measurements in this report.

12. Ethical Considerations and Limitations

12.1 Copyright and Intellectual Property

All data in the RAG knowledge base is drawn from openly licensed materials. OWASP content is used under Creative Commons Attribution-ShareAlike. CWE is public domain. Style guide excerpts are either public documents or summaries authored for this project. No proprietary code or corporate documents are included.

12.2 Bias and Representation

The evaluation suite covers two languages (Python and JavaScript) and CWE categories that skew toward web application and backend concerns. The RAG corpus additionally indexes Java-specific patterns and the synthetic generator includes Java templates, but no Java sample appears in the ten-sample hand-labeled toy suite or in the twenty-sample paired suite; the quantitative claims in Section 10 are therefore bounded to Python and JavaScript. The system is likely weaker on systems languages such as C or Rust, on embedded contexts, and on domains such as smart contracts that have their own vulnerability taxonomies. A Java evaluation pass against a hand-labeled Java suite is a concrete follow-on task, sized against the same 60-to-80-paired-sample power-analysis target computed in Section 10.9.

12.3 Misuse Scenarios

A code review system capable of identifying vulnerabilities is also, in principle, capable of being used to find vulnerabilities in code the user does not own. The synthetic data pipeline explicitly avoids producing weaponized exploit code, and the system's outputs are framed as remediation-oriented findings rather than exploit instructions. Emitting a concrete fix alongside every finding is itself a defensive posture: the remediation is the intended consumable, not the exploit.

12.4 Limitations

Three limitations deserve explicit acknowledgment. The system depends on an LLM whose behavior is not fully deterministic. The evaluation suite at ten hand-labeled samples is modest; confidence intervals on the reported metrics are wide, as detailed in Section 10.5. The system is not a substitute for formal security review in regulated contexts such as payment systems or medical devices, where human accountability and traceable review processes are mandatory.

13. Future Work

Extensions fall into three tiers: near-term engineering, medium-term research, and long-term positioning as either a production system or a standalone startup. Each tier has concrete scoping numbers derived from this report's measurements so that follow-on work is not speculative.

13.1 Near-Term Engineering Extensions

First, wiring the reinforcement learning modules described in Section 8 into the production agent graph so that prompt-variant selection and post-rejection routing become learned policies rather than fixed choices. The infrastructure exists; what is missing is the connection to the live decision points and an ablation study measuring the contribution of each policy against the fixed-prompt baseline reported in Section 10. Second, extending the Semgrep comparison reported in Section 10.10 to codebases with known vulnerabilities — Flask 0.12.x, an intentionally vulnerable teaching repository, a small Django application — to produce a side-by-side detection comparison rather than the zero-zero agreement on clean code observed on April 20, 2026. Third, expanding the hand-labeled toy suite to at least one hundred samples, which would reduce confidence interval width to usable ranges and enable statistical significance on the toy-suite effect size; Section 10.9 power analysis estimates this at 60 to 80 paired discordant-pair samples, so the 100-sample expansion is appropriately sized. Fourth, running a dedicated Java evaluation pass against a hand-labeled Java suite, since the RAG corpus indexes Java patterns and the synthetic generator includes Java templates but the current Section 10 evaluation does not exercise them. Fifth, integration with a continuous integration platform (GitHub Actions or GitLab CI) to move the system from advisory to gating on unresolved critical findings. Sixth, replacing REINFORCE with a Deep Q-Network over the complete agent graph, which would close the loop on treating the multi-agent system as a learnable object.

13.2 A Production-Grade Five-Plus-Agent Architecture

The three-agent architecture shipped in this release is the minimum-viable-logic specified in the February proposal's risk register. A production system addressing a larger codebase or a broader class of defects would benefit from additional specialist agents, each with a narrow mandate. The architecture generalizes naturally because all agents share the same typed state and all agent outputs flow through the same Evaluator Guardian gate. Adding agents does not require redesigning the graph; it requires adding nodes, extending the state schema, and extending the Evaluator's rejection conditions.

Proposed production agent roster (five-plus)

Security Sentinel — unchanged. RAG-grounded vulnerability detection over OWASP Top 10, CWE, and language-specific patterns. Code Quality Auditor — unchanged. Style, readability, maintainability, capped at ten findings. Performance Analyst — new. Detects obvious performance pitfalls (N+1 queries, synchronous I/O in hot paths, unbounded memory allocation). Grounded in a separate RAG corpus of performance anti-patterns. Architectural Synthesizer — new. Reconciles cross-agent findings when they disagree (e.g., a cache that Security flags as MD5 misuse and Performance flags as missing). Produces a unified finding with a negotiated verdict. Threat Modeler — new. Given the code and its imports, produces a coarse-grained threat model (attacker entry points, trust boundaries, data flows).

Security Sentinel findings are re-scored against this model so that a cryptography bug in a public endpoint is prioritized over the same bug in an internal tool. Compliance Auditor — new. Maps findings to regulatory frameworks (SOC 2, HIPAA, PCI-DSS) where applicable. Only emits findings when a regulatory frame is configured. Evaluator Guardian — unchanged role, expanded rejection conditions. Now also validates Architectural Synthesizer negotiations and Threat Modeler consistency.

Why five-plus, not five exactly?

The "plus" is deliberate. A production architecture should be open-ended in the number of specialists so that new defect classes can be added without reorganizing the graph. The invariants are: one grounded retrieval step per specialist, one schema per specialist, one Evaluator gate that all findings pass through before user-visible output. Adding a seventh agent (e.g., a Dependency-Security Agent that queries GitHub Advisory Database) requires a new RAG corpus and a new schema, but does not require touching the Evaluator's Layer 1 programmatic checks, because those operate on schema-conformant findings regardless of which agent produced them.

13.3 Positioning as a Claude Mythos / Project Glasswing Alternative at Open-Tool Scale

Two weeks before the submission of this report, Anthropic disclosed Claude Mythos Preview and the associated Project Glasswing initiative, in which the same vendor that produces the underlying model used by this project deploys that class of model to find zero-day vulnerabilities in critical software at industrial scale. The public description of Project Glasswing is instructive because its architectural pattern is structurally similar to the architectural pattern explored here: files or code units are ranked by likelihood of containing bugs before analysis, specialized agent instances analyze different units in parallel, and a final, independent agent pass is used to confirm which findings are real and interesting before any report is surfaced to a human.

That pattern, at Project Glasswing scale, is finding thousands of zero-day vulnerabilities in operating systems and web browsers. At the scale of this project, the same pattern, with an adversarial Evaluator substituted for the final confirmation agent and a curated RAG corpus substituted for autonomous code execution, is finding textbook vulnerabilities on a ten-sample suite and doing so with a measurable architecture-attributable improvement over both an out-of-the-box baseline and a prompt-iterated baseline on the identical model.

The positioning claim

CodeSentinel does not compete with Project Glasswing on capability. It demonstrates, within the bounds of an academic project, that the architectural pattern in use at the frontier is reproducible, testable, and teachable with open tools. A future production-scale extension would differ from Glasswing in three respects: (a) it would run locally against the user's own codebase rather than as a vendor service, which matters for organizations that cannot send source to a third party; (b) it would use open-source components (LangGraph, ChromaDB, sentence-transformers) rather than proprietary infrastructure; (c) it would produce grounded, citation-backed findings rather than autonomous exploit confirmations, which lowers the deployment bar by two orders of magnitude and removes the need for sandbox execution infrastructure. This is a complementary positioning — a "Glasswing for organizations that cannot send code to Anthropic" — not a competitive one.

A scaled-up CodeSentinel operating in this role would need three capabilities beyond the current release. First, file-ranking: a cheap pre-pass that scores each file in a repository by predicted-defect-likelihood, using a smaller model than the primary analyst agents, so that analysis spend is concentrated on the top-K files rather than linearly across the codebase. Second, parallel agent instances: at scale, multiple Security Sentinel instances running concurrently on different files, with findings merged through the Architectural Synthesizer. Third, a confirmation mode modeled loosely on Project Glasswing, in which the system can optionally execute test harnesses in a sandbox to confirm hypothesized vulnerabilities before surfacing them. The architecture itself does not need to change to accommodate any of these; that is the point of the pattern.

13.4 Research Extension

Several research questions fall naturally out of the measurements in Section 10. The three-way decomposition in Table 2 — where roughly 60% of the false-positive reduction is attributable to prompt engineering and 40% to architecture — is a single measurement against a specific baseline and model. The generalization question is whether the architecture-attributable share persists (a) across model families (Claude Sonnet vs. GPT-4 vs. open-source models), (b) across task domains (code review vs. documentation review vs. academic peer review), and (c) across defect densities (ten-sample toy suites vs. hundred-sample enterprise codebases). A natural PhD-scale research program would test the hypothesis that the specialize-ground-adversarially-review pattern contributes a stable residual reliability gain above whatever prompt-engineering ceiling exists for a given model.

Three concrete research extensions are sized for a 12 to 18 month timeline. First, a 1000-sample paired-evaluation suite across five languages (Python, JavaScript, Java, Go, Rust) would reduce confidence interval width to sub-5% and enable cross-language analysis of where the architecture helps most. Second, an ablation study systematically disabling each component of the architecture (retrieval only, Evaluator only, retrieval plus Evaluator, etc.) to attribute the architectural residual to its constituent mechanisms. Third, an agent-meta-learning study treating the agent graph itself as a learnable object: a higher-level policy that adds or removes agent nodes in response to evaluator rewards over thousands of samples, which is a natural generalization of the REINFORCE routing policy in Section 8.

13.5 Standalone Startup / Commercialization Path

The architecture has plausible commercialization pathways, listed here for completeness and framed honestly about where the bar is. The primary market opportunity is as a pull-request gate SaaS: CodeSentinel runs on every PR opened against a customer's repository, posts findings as inline comments, and blocks merge on unresolved critical findings. The technology in this report is roughly 30% of the product; the other 70% is GitHub App integration, enterprise authentication and single-tenant data handling, per-customer RAG corpus customization, a finding-triage workflow, and pricing/contract infrastructure.

Why "complementary, not competitive" is the right positioning

Semgrep, CodeQL, and Snyk have a decade-plus head start and dominate the SAST market. Trying to beat them at rule-coverage breadth is a losing bet. The correct positioning is that CodeSentinel covers

> a different failure mode: context-dependent defects requiring grounded reasoning that rule-based tools cannot express. A CI pipeline in 2027 plausibly runs Semgrep first (fast, free, high-recall on rule-specified patterns), then routes Semgrep-unique findings and non-findings to CodeSentinel for deep review (slow, costs API credit, but gives remediation text and CWE grounding that Semgrep does not). The market is additive, not substitutive, which means the sales motion is "add us alongside your existing SAST" rather than "rip and replace," which is an order of magnitude easier.

A viable go-to-market for an early-stage company would target three customer segments in order: (1) regulated-industry engineering teams (fintech, healthtech, defense) that cannot send source code to third-party AI vendors for security review and therefore cannot use Anthropic's Project Glasswing service or equivalent; these teams have budget and procurement infrastructure but are blocked on privacy. (2) Mid-market engineering teams that already use Semgrep or Snyk and experience high false-positive rates; these teams have the pain (alert fatigue) but are price-sensitive. (3) Open-source foundations with critical infrastructure exposure (Apache, Linux Foundation projects); these teams have the visibility but not the budget, so serving them is a marketing investment rather than a revenue line.

Concrete initial product milestones would be: an open-source self-hosted version (what this repository approximates) serving as top-of-funnel; a managed-service version with GitHub App integration and seat-based pricing at 25 to 50 US dollars per developer per month; and an enterprise tier with single-tenant deployment, customer-specific RAG corpus augmentation, and SOC 2 attestation. The per-finding API cost measured in this report (roughly 0.02 to 0.05 US dollars per sample at Claude Sonnet prices) means the gross margin on a 50 dollar per developer per month plan is positive so long as usage stays under approximately 1,000 findings per developer per month, which matches typical SAST-tool usage patterns. Unit economics are viable; the binding constraint is sales and enterprise-contracting motion, not technology.

13.6 The Teachable-Pattern Claim

Independently of whether this project becomes a product, research program, or neither, the core contribution of this report is a demonstration that the specialize-plus-ground-plus-adversarially-review pattern, which is being deployed at the frontier by the same vendor whose model this project uses, can be reproduced, measured, and taught on open tools. A Northeastern Information Systems graduate, working for a semester with a laptop, an Anthropic API key, and publicly available libraries, can produce a measurable reliability gain that goes beyond what careful prompt engineering alone can achieve on a known LLM failure mode. That the same pattern at the frontier produces qualitatively different capabilities (zero-day discovery at industrial scale) is not a contradiction; it is a statement about the relationship between architecture and scale. The architecture is necessary for reliability at any scale, and it compounds with capability at larger scales. Treating it as a first-class engineering pattern — rather than as a wraparound on top of a prompt — is the lesson this report contributes to the course.

14. Conclusion

CodeSentinel demonstrates that a small, disciplined multi-agent generative AI system can outperform both an out-of-the-box single-prompt baseline and a carefully iterated single-prompt baseline on code review. The three contributions driving this result are specialization via agent roles, grounding via retrieval-augmented generation against authoritative sources, and adversarial self-critique via a dedicated Evaluator Guardian with both programmatic and LLM validation. The project additionally contributes a synthetic data generation pipeline with independent verification, and a parallel reinforcement learning demonstration module that is not integrated into the agent graph in this release and does not contribute to the reported benchmark numbers.

The central empirical result

From the April 20, 2026 real-LLM run on the ten-sample toy suite: all three systems achieve perfect recall. The out-of-the-box single-prompt baseline produces thirty false positives. The prompt-iterated baseline, having received three rounds of refinement on the same held-out set used for the agent prompts, reduces this to twelve. The multi-agent pipeline produces one. Roughly 60% of the false-positive reduction is attributable to prompt-engineering effort any careful practitioner could replicate; the remaining 40% is attributable to the retrieval-and-adversarial-review architecture. Both contributions are real; the architecture is a complement to careful prompt engineering rather than a substitute for it. This is the most direct measurement of the Evaluator Guardian's contribution once prompt-engineering effort is controlled for, and it is the primary outcome on which the project stands.

Mock-mode runs (CI-reproducible without API keys) corroborate the direction on the toy suite (multi-agent 1.000 TPR versus baseline 0.750) and provide statistically significant evidence on the twenty-sample paired suite (McNemar's exact $p = 0.0312$, significant at $\alpha = 0.05$). CWE classification accuracy is 1.0 across all suites and all 35 unit tests pass on a clean clone. The Semgrep comparison on April 20, 2026 returned zero findings from both systems on well-maintained Flask production code, establishing that the multi-agent pipeline is not over-triggered on clean inputs. The confidence intervals around these point estimates are wide — a 95% Wilson interval on the multi-agent FPR spans roughly 0.02 to 0.43 — which is why the 100-sample expansion in Section 13.1 is the right follow-on rather than further spend on the current suite.

The underlying claim is broader than the specific system: when an LLM application is rearchitected into specialized, grounded, adversarially-reviewed agents, the gains go beyond what prompt-engineering labor alone can buy, and they compose with any model the field produces next. That claim is what the measurements in Section 10 establish, within the bounded scope specified by the non-objectives in Section 2.3. The extensions proposed in Section 13 span near-term engineering, a five-plus-agent production architecture, research questions sized for a 12-to-18-month program, and a concrete commercialization path — all of which follow from the same architecture without requiring it to be redesigned.

Project Artifacts and Companion Links

Every claim in this report is backed by an artifact the reader can inspect independently. Four companion resources are published alongside the repository. The project website at

aravindb98.github.io/CodeSentinel hosts the showcase page with the same warm-orange and sindoor palette used throughout this report. The GitHub repository at github.com/AravindB98/CodeSentinel contains all Python source files, the 35-sample test suite, the RAG corpus, the RL demonstration modules, the benchmark harness, and the committed April 20, 2026 real-LLM results under eval/results/20260420_143220/. The Streamlit deployment at codesentinel-f2ggdvqeuwsj4pta5sk27s.streamlit.app runs the full three-agent pipeline against real Claude Sonnet, with a streaming intermediate-results UI (st.status containers updated per LangGraph node) so the user sees per-agent progress as the graph executes rather than a blank screen while Evaluator retries run — per-call costs run on a personal API account as disclosed in Section 11.7. The YouTube video at youtu.be/do8GvAK7tHI is a thirteen-minute walkthrough covering the architecture, the live Streamlit demo including the streaming UI, the three-way comparison result, and the bug the system caught before the author did.

15. References

[1] OWASP Foundation. OWASP Top 10 2025. https://owasp.org/Top10/

[2] MITRE. Common Weakness Enumeration (CWE). https://cwe.mitre.org/

[3] Chase, H. LangChain and LangGraph documentation. https://python.langchain.com/docs/langgraph

[4] Reimers, N., Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

[5] Lewis, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[6] Auer, P., Cesa-Bianchi, N., Fischer, P. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning 47, 2002.

[7] Williams, R. J. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. Machine Learning 8, 1992.

[8] van Rossum, G., Warsaw, B., Coghlan, A. PEP 8: Style Guide for Python Code.

[9] Google. Google Python Style Guide. https://google.github.io/styleguide/pyguide.html

[10] Airbnb. Airbnb JavaScript Style Guide. https://github.com/airbnb/javascript

[11] Anthropic. Project Glasswing: Securing Critical Software for the AI Era. April 2026. https://www.anthropic.com/glasswing

[12] Anthropic Frontier Red Team. Claude Mythos Preview: Technical Notes on Vulnerability Discovery. April 2026. https://red.anthropic.com/2026/mythos-preview/

[13] Veracode. State of Software Security 2025. 2025.

[14] Stack Overflow. 2025 Developer Survey. https://survey.stackoverflow.co/2025/

[15] Codacy. Code Review Process: Impact on Developer Productivity. 2025.

[16] IBM Security. Cost of a Data Breach Report. 2023. Global average: $4.45 million per incident.

[17] McNemar, Q. Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika 12(2), 1947.

[18] Wilson, E. B. Probable Inference, the Law of Succession, and Statistical Inference. JASA 22(158), 1927.

[19] Youden, W. J. Index for rating diagnostic tests. Cancer 3(1), 1950.

[20] OWASP Foundation. OWASP Benchmark Project. https://owasp.org/www-project-benchmark/

[21] OWASP-Benchmark GitHub organization. BenchmarkJava. https://github.com/OWASP-Benchmark/BenchmarkJava

[22] Semgrep Inc. Semgrep: Static analysis at ludicrous speed. https://semgrep.dev

[23] GitHub. CodeQL: Semantic code analysis engine. https://codeql.github.com

[24] Snyk Ltd. Snyk Code: AI-powered SAST. https://snyk.io/product/snyk-code/

[25] Bito AI. CodeReviewAgent. https://bito.ai/ai-code-review-agent/

[26] Tabnine. Code Review Agent. https://www.tabnine.com/code-review-agent/

Appendix A. Prompt Template Excerpts

A.1 Security Sentinel System Prompt (excerpt)

```
You are a senior application security engineer. Your only job is to identify
security vulnerabilities.  Rules:   1. Every finding MUST cite a retrieved
passage in rag_source.      Findings without a citation are invalid.   2. If
you are uncertain whether code is vulnerable, suppress      the finding
rather than invent one.   3. Output valid JSON matching the SecurityFinding
schema.
```

A.2 Evaluator Guardian System Prompt (excerpt)

```
You are an adversarial reviewer. Assume the upstream agents have made
mistakes.  Reject any finding that meets ANY of these:   - rag_source empty
or not in retrieved context   - evidence does not point to actual code lines
- fix is vague or not implementable   - schema is malformed   - two findings
contradict each other
```

Appendix B. Per-Sample Benchmark Results

Per-sample outcomes on the ten hand-labeled samples, from the April 20, 2026 real-LLM run. BL-OOB is the out-of-the-box baseline; BL-ITER is the prompt-iterated baseline; MULTI is the multi-agent pipeline. TP columns count true positives; FP columns count false positives.

Sample	Lang	Truth	BL-OOB TP / FP	BL-ITER TP / FP	MULTI TP / FP
TOY-001	Python	SQL	1 / 3	1 / 1	1 / 0
TOY-002	Python	Clean	0 / 2	0 / 1	0 / 0
TOY-003	Python	Pickle	1 / 4	1 / 2	1 / 0
TOY-004	Python	CmdInj	1 / 3	1 / 1	1 / 0
TOY-005	Python	MD5	1 / 3	1 / 1	1 / 1
TOY-006	Python	TLS	1 / 3	1 / 1	1 / 0
TOY-007	Python	Clean	0 / 3	0 / 2	0 / 0
TOY-008	JS	SQL	1 / 3	1 / 1	1 / 0
TOY-009	JS	XSS	1 / 3	1 / 1	1 / 0
TOY-010	Python	yaml	1 / 3	1 / 1	1 / 0
Totals			**8 / 30**	**8 / 12**	**8 / 1**

The two samples on which the systems diverge most are TOY-005 (hashlib.md5 for password hashing) and TOY-010 (yaml.load without SafeLoader). Both require recognizing a language-specific pattern that the out-of-the-box baseline does not reliably surface. The multi-agent system's RAG retrieval step supplies the PY-07 (yaml.load) and PY-08 (Python MD5/SHA1) patterns directly to the Security Sentinel, which is why the multi-agent catches both. TOY-005 is the only sample where the multi-agent system carries one false positive (the MD5-as-cache-key confusion analyzed in Section 10.8).

Appendix C. Single-Prompt Baselines (verbatim)

C.1 Out-of-the-Box Baseline

The baseline single prompt compared against the multi-agent system in Section 10 is reproduced verbatim below, from eval/baseline_single_prompt.py. This is the exact system prompt used in the April 20, 2026 real-LLM benchmark run that produced thirty false positives. The baseline is deliberately competent, not a straw man: it specifies the same output schema as the multi-agent system, requires the same CWE/OWASP fields, and demands concrete remediation. The only architectural difference is that it does not retrieve against the RAG corpus and does not run an adversarial Evaluator pass.

```
BASELINE_SYSTEM_PROMPT = """ You are a code reviewer. Analyze the user's code
for security vulnerabilities. Return a single JSON object with one top-level
key "findings" whose value is an array.  Each finding MUST have:   finding_id
(string "SEC-001"),   category,   cwe_id ("CWE-N"),   owasp_ref,   severity
(CRITICAL / HIGH / MEDIUM / LOW),   confidence (0.0-1.0),   evidence { file,
line_start, line_end, snippet },   fix (concrete remediation, at least 20
chars),   rag_source { doc, passage_id }.  No prose. No markdown fences. JSON
only. If no issues, return {"findings": []}. """
```

The user-turn template wrapped around each evaluation sample is: LANGUAGE: <python|javascript|java>, followed by the source code in a fenced block, followed by the instruction "Analyze the code and return the findings JSON now." Temperature is set to 0.0 and max_tokens to 3000, matching the multi-agent Security Sentinel call. The baseline's rag_source field is emitted by the model from parametric memory rather than retrieved, which is precisely the failure mode the multi-agent architecture's citation-verification is designed to catch: the Evaluator programmatically rejects findings whose rag_source is not in the retrieved context.

C.2 Prompt-Iterated Baseline

The prompt-iterated baseline begins from the same V1 prompt and receives three rounds of refinement against the same six-sample held-out validation set used to tune the agent prompts in Section 5.1. Changes introduced across the three rounds include explicit negative examples targeting the specific false-positive classes observed in V1 runs (parameterized queries flagged as SQL injection, constant strings flagged as user-controlled input, standard library calls flagged as vulnerabilities), a minimum-confidence threshold of 0.6 below which findings must be suppressed rather than emitted, and a requirement that the fix field be at least twenty characters of implementable instruction rather than a generic phrase. This is the baseline whose twelve-false-positive result appears in Table 2 of Section 10.4.1.

```
BASELINE_SYSTEM_PROMPT_ITER = """ You are a code reviewer. Analyze the user's
code for security vulnerabilities.  Rules:   1. Do NOT flag parameterized
queries using placeholders as SQL      injection. Concatenation or f-string
interpolation into SQL      IS injection; placeholders are NOT.   2. Do NOT
flag constant strings as user-controlled input.   3. Do NOT flag standard
library calls without specific context      indicating misuse (e.g.,
hashlib.sha256 is not inherently      a vulnerability; hashlib.md5 in a
password context is).   4. If you are not at least 0.6 confident, omit the
finding.      A missed finding is better than a fabricated one.   5. Every
finding must include a concrete, implementable fix      (at least 20
```

```
characters). "Use a safer function" is not a      fix; "Replace hashlib.md5
with hashlib.sha256 and add a      per-user salt stored in env" is.  [...
same output schema as V1 above ...] """
```

The gap between the V1 prompt (30 false positives) and the iterated prompt (12 false positives) is the prompt-engineering contribution. The gap between the iterated prompt (12) and the multi-agent system (1) is the architectural contribution. Reporting both is what allows the decomposition in Section 10.4.1 to be made honestly rather than as an implicit assumption that all observed improvement was architectural.